From Tools to Missions: A Mission-Capability Runtime for Enterprise AI

Credible Lineage, Measurable Post-LLM Capability Scoping, Engineering Proof, and a Prerequisite-Conditioned Long-Horizon Roadmap

Prasad Deshpande

Independent Researcher – Agentic AI Systems and Enterprise Architecture

0009-0008-1344-6585 er.prasad.deshpande@gmail.com prasaddeshpande.com

Adversarially Hardened Whitepaper Candidate R1.6.1 – 31 August 2026

Publication-layer provenance. This whitepaper is derived from the frozen scientific manuscript (SHA-256 `E589393C6EEA0BA92E4EC521D1577A6F561F9ADFA38FF9EEE97A94695C666D90`) and the engineering-proof editions that followed it. No experiment was rerun, no raw evidence was modified, and no empirical claim was strengthened. R1.6.1 is a bounded post-adversarial-review hardening release: it preserves the R1.6 architecture, figures and publication layout while narrowing claim/evidence boundaries, completing LLM-era tool-retrieval lineage, and clarifying maturity labels. The OWL-S/WSMO/SOA/Saga/Temporal ancestry remains conceded; provider independence remains a binding/runtime property rather than an MCWS result; and CURRENT, EMERGING, ARCHITECTURE HYPOTHESIS and LONG-HORIZON material remain explicitly separated. Future constructs remain a prerequisite-conditioned research roadmap, not experimental results.

Abstract

Goal-driven service discovery, semantic capability descriptions, durable workflow state, compensation and dynamic provider binding have deep ancestry in Semantic Web Services, SOA/BPM and durable-workflow systems. LLM-era tool-retrieval and dynamic-selection work also predates this release and already treats the model-visible tool set as an important reasoning input [1, 2, 3, 4]. The narrower problem examined here is how an enterprise runtime should make executable capability exposure *mission-stage-specific* while keeping authorization, final provider ranking and execution authority outside probabilistic model control. This paper evaluates stage-scoped capability exposure through a Minimum Capability Working Set (MCWS). The frozen corrected primary result is unchanged: MCWS achieved 30/48 mission successes (62.5%), Static 17/48 (35.4%) and Broad 5/48 (10.4%); Holm-adjusted case-cluster tests were $p = 0.00293$ for Broad–MCWS, $p = 0.03125$ for Static–MCWS and $p = 0.2266$ for Broad–Static.

Building on that bounded result and separate SAP/MCP/Capella/BAS engineering proofs, R1.6.1 *pairs* the evidenced MCWS mechanism with a historically grounded practitioner Mission-Capability Runtime pattern: durable mission state produces stage capability demand; model-visible capability exposure is bounded; deterministic runtime logic validates the capability requirement and owns tenant/policy/trust admissibility, final provider ranking and binding; and authoritative enterprise observation can trigger mission-state transition and recomposition. The full pairing is architecture synthesis, not a system-level result established by the frozen experiment. The paper does **not** claim a new computational paradigm or capability-contract standard; its four runtime records remain an operational schema and future constructs remain prerequisite-conditioned hypotheses.

1 Executive summary

The 2026 enterprise-agent market has moved quickly. MuleSoft now ships an *Agent Fabric* with registry, governance, orchestration and observability; Google operates an Agent Registry plus Agent Gateway; AWS combines AgentCore Gateway with deterministic Cedar policy; Microsoft Agent 365 synchronizes agents across external platforms; SAP Integration Suite exposes APIs, HTTP endpoints and RFC-based SAP backends as governed MCP server artifacts [5, 6, 7, 8, 9]. Meanwhile, research systems such as Queen-Bee and Tool Forge already combine capability retrieval, constrained tool access, governed catalogs and scoped routing [10, 11].

That convergence changes the whitepaper's claim. The differentiator is **not** another "fabric," another capability-contract vocabulary, or another registry. R1.6 separates four contribution levels so historical lineage, measured evidence and future architecture are never conflated.

The practical contribution is therefore narrower:

1. represent a mission independently of provider/tool identity;
2. resolve a provider-neutral capability set for the current mission stage;
3. expose only that stage's working set to model reasoning;
4. re-resolve when provider or mission state changes; and
5. keep business authorization and outcome truth at deterministic enterprise boundaries.

Decision rule. Use static execution when the mission path and provider binding are stable. Introduce a mission-capability runtime when provider substitution, requirement change, heterogeneous backends, delegated authority, or long-running enterprise state makes runtime resolution valuable.

Table 1. R1.6 contribution stack: what is evidenced, engineered, synthesized and hypothesized.

Contribution level	What this paper contributes	Evidence class	Explicit boundary
Scientific	Stage-scoped MCWS evaluation under Broad/Static/dynamic mission conditions	Frozen experiment	Does not establish universal dynamic-orchestration superiority.
Engineering	Mission contracts, live ADT MCP access, Capella/A4Layer orchestration, SAP lifecycle/readback, BAS/Fiori consumer path	Separate frozen POCs	POCs demonstrate surrounding components, not additional experiment observations.
Architecture synthesis	Mission demand → bounded reasoning surface → deterministic admissibility/binding → outcome evidence/recomposition	Practitioner pattern	Builds on OWL-S/WSMO/SOA/Saga/Temporal ancestry; not claimed as a new computational primitive.
Long-horizon research agenda	Portable missions, trust federation, economic routing, digital-twin/cyber-physical extensions	Hypothesis / scenario	Prerequisite-conditioned and non-empirical.

Authority rule. The LLM may interpret ambiguous mission semantics and propose capability intent; the deterministic runtime owns admissibility, provider ranking, binding integrity and execution authority. In short: **the LLM proposes semantics; the runtime binds authority.** Because an LLM-authored capability requirement can still shape candidate eligibility, the runtime must validate that requirement against durable mission state, allowed capability classes, schema, tenant, policy and delegated authority before provider eligibility or ranking.

The paper's empirical evidence supports that narrower claim: after common-target correction, MCWS achieved materially higher primary mission success than both Broad and Static in the evaluated benchmark; Static remained adequate on stable/composition families but was not empirically superior there.

2 What we have already built: engineering proof at a glance

R1.6 preserves two evidence streams. The **research stream** is the frozen 180-condition Broad/Static/MCWS study. The **engineering stream** is a set of separately frozen SAP/Capella/MCP/BAS POCs built before and alongside the research program. The engineering POCs are *not* retroactively counted as experiment observations; they demonstrate that surrounding architectural components have been exercised against real enterprise development/runtime surfaces.

Engineering claim boundary. The paper does not claim that one monolithic runtime executed every proof in a single run. The proof register maps independently frozen implementation evidence to architecture components. Different packages and generations remain distinct; for example, historical `ZCLINE` proofs are not conflated with later `ZA4CLAYER` compiler work.

2.1 Evidence-to-architecture map

3 Independent-review alignment: what changed

The R1.1 architecture expansion survived independent review, but the review exposed four important collisions with the 2026 landscape. R1.2 acted on all four; R1.3 keeps those boundaries unchanged and adds engineering evidence without expanding the empirical claims.

4 Where the runtime fits

4.1 Protocol, control plane and mission runtime are different layers

The strongest architecture is compositional rather than monolithic. Figure 2 separates five concerns.

MCP remains the vertical interoperability boundary for tools/resources/tasks. A2A is adjacent for agent-to-agent collaboration. Registries and gateways discover, mediate and govern endpoints. Business systems remain the final authority for actual permissions and committed state. The runtime sits above these layers and answers a different question: *given the mission's current state, which capability is required now, which provider is qualified to deliver it, and what should happen if that binding becomes invalid?*

4.2 Mission state, not provider identity

The core model remains:

$$\text{Mission} \rightarrow \text{Capability} \rightarrow \text{Provider} \rightarrow \text{Tool}.$$

The provider can change without changing the mission. This is the key architectural separation retained from the technical report.

5 The measurable post-LLM problem: capability catalogs become reasoning input

The historical lineage matters because the paper's distinctive problem is not service composition itself. OWL-S/WSMO-era systems already modeled goals, capabilities, service grounding, preconditions/effects and dynamic composition. The post-LLM operational change is that human-readable tool/capability descriptions can be placed directly inside a generative model's active context. The *catalog presented to the model* therefore becomes part of the model input and can alter selection behavior even when the underlying enterprise services are unchanged.

A broad catalog is therefore not just a discovery database. It can be an executable reasoning menu. This creates a specific control problem: capability exposure must be small enough to reduce semantic collisions and attack surface, yet complete enough to preserve the mission's current obligations. R1.6.1

Table 2. Engineering proof register at a glance. Proof states are intentionally conservative.

ID	Architecture component	Observed implementation evidence	Proof state	Boundary
P1	Provider-independent mission contract	Mission 001 read-only implementation path; target `TRL_EN`, client 100; provider-independent semantics and explicit no-mutation constraint	**VALIDATED-READ-ONLY**	Architecture mission only; no repository/runtime mutation required.
P2	MCP target/provider access	ADT MCP on port 2236 exercised against `TRL_EN`; repository/service capabilities available; controlled replication recorded zero ADT-down samples and zero SAP writes	**VALIDATED-READ-ONLY**	ADT MCP is the strongest live MCP proof; Fiori/UI5 MCP roles are mapped but not claimed as equally live-qualified in the same execution path.
P3	Mission-aware SAP execution chain	Capella orchestrates; A4Layer executes; SAP remains system authority; generated RAP/OData/Fiori lifecycle separated from orchestration authority	**PROVEN-LIVE**	Separate engineering implementation, not part of EA7/EA8 experimental cells.
P4	Full-stack enterprise consumer path	Frozen full-stack E2E milestone; app `ZA4CLANG0010`; activation, service publication, OData V4, Fiori Elements, BAS build, SSO, BSP deployment and rendered OData	**PROVEN-LIVE**	Historical frozen proof; the app is not reused or modified for this paper.
P5	Bounded execution / no arbitrary shell authority	BAS production runner uses a four-operation whitelist, zero automatic retry, frozen toolchain, explicit Fiori intent gate, and evidence-only closure separation	**IMPLEMENTED**	Supports bounded executor design; not a security certification.
P6	Mission-capability selection/recomposition	Corrected Broad/Static/MCWS study: MCWS 30/48 vs Static 17/48 vs Broad 5/48; common-target reanalysis and cluster statistics	**EXPERIMENTALLY-EVALUATED**	Read-only authored benchmark; no universal production claim.
P7	Enterprise failure semantics	SAP activation, projection semantics, provider availability, generated-object identity, and post-deploy tooling warnings were preserved as distinct failure classes rather than collapsed into "agent failed"	**PROVEN-LIVE**	Observational engineering evidence; frequencies are not population estimates.

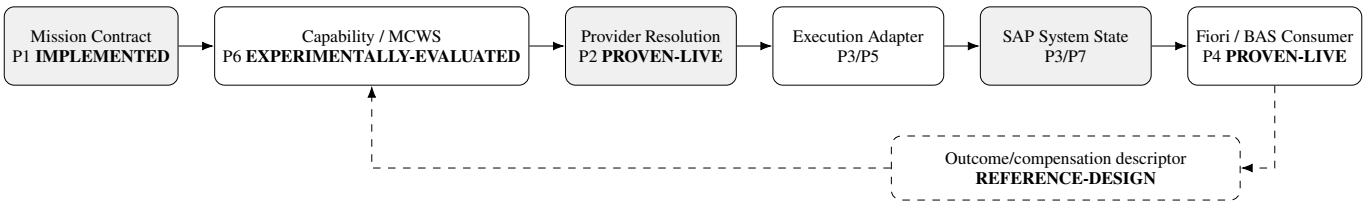


Figure 1. Evidence map. MCWS/recomposition is experimentally evaluated; SAP/MCP/BAS components are supported by separate engineering POCs. Outcome/compensation semantics remain reference design.

treats this as the primary post-LLM role of MCWS. **Attribution boundary.** MCWS manages the model-visible reasoning surface. Provider abstraction, provider ranking, provider substitution, credentials, binding integrity and execution authority belong to the deterministic Mission-Capability Runtime; they are not credited to MCWS itself.

6 Practical implementation profile

R1.3 retains the R1.2 decision to remove the proposed "Enterprise Capability Contract" as a headline construct because portable capability declarations now exist independently. MCP itself already exposes risk-oriented annotations such as read-only, destructive and idempotent hints, while ACC defines an emerging operation-level declaration model for scope, risk, subject, approval and audit metadata [16, 13]. These should be reused where appropriate rather than duplicated.

What the mission runtime still needs is a **runtime resolution object**: a local projection that joins mission state to provider metadata. The following is illustrative implementation data, not a proposed standard:

```
{
  "mission_id": "PO-RELEASE-2026-001",
  "stage": "diagnose",
  "required_capability": "purchase_order.read",
  "candidates": [
    {
      "provider": "sap-is-rfc-mcp-v1",
      "tool": "BAPI_PO_GETDETAIL1",
      "source_digest": "sha256:...",
      "risk": "read_only",
      "authority_ref": "policy://procurement/read",
      "health": "qualified"
    },
    {
      "provider": "sap-api-mcp-v2",
      "tool": "PurchaseOrder.Read",
      "source_digest": "sha256:...",
      "health": "standby"
    }
  ],
  "selected_provider": "sap-is-rfc-mcp-v1",
  "outcome_check": "purchase_order.readback"
}
```

The runtime should never treat declaration metadata as final business authorization. ACC itself makes the same reach-versus-

Table 3. R1.2 differentiation after independent review and market verification.

R1.1 framing	2026 overlap verified	R1.2 treatment
Enterprise Capability Fabric as headline	MuleSoft Agent Fabric already uses registry + governance + orchestration + observability; IBM and Microsoft also market control-plane/fabric-like layers [5, 12, 8]	"Fabric" becomes market/infrastructure context, not the paper's novelty. Headline contribution becomes the Mission-Capability Runtime pattern.
Enterprise Capability Contract	Agent Capability Contract (ACC) already defines a portable business-capability declaration model; Tool Forge uses a capability-contract intermediate representation [13, 11]	Do not invent a competing contract. Compose MCP annotations, ACC-like governance metadata, provider catalogs and business-system authority into a runtime resolution view.
Governed capability retrieval / scoped execution	Queen-Bee retrieves capabilities into BeeSpec and evaluates governed scoped execution; Tool Forge performs intent-scoped MCP routing [10, 11]	Treat as close prior art. Distinguish mission-state-driven recomposition, provider-landscape evaluation and the corrected Broad/Static/MCWS study.
Mission semantics and authority	Mission-Bound Authorization/Runtime/Capability Binding Internet-Drafts make mission authority and capability-source binding explicit [14, 15]	Clarify that this paper's mission contract is orchestration state, not an authorization credential. Authorization remains a complementary enforcement layer.

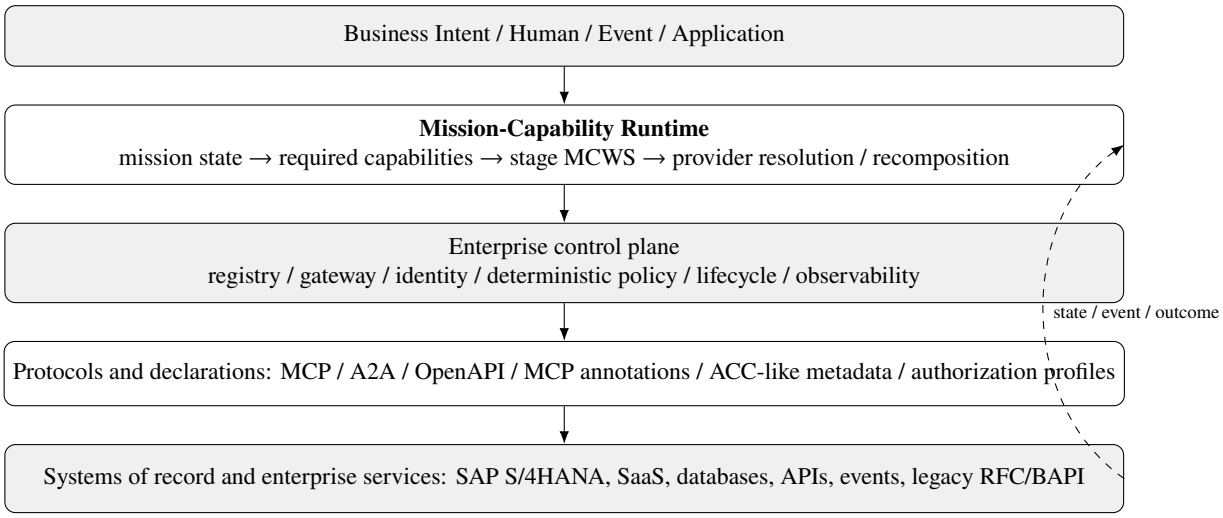


Figure 2. R1.2 reference stack. The Mission-Capability Runtime is not a replacement for MCP, gateways, capability declarations, or enterprise authorization. It consumes those layers to keep mission semantics stable while bindings change.

authority distinction, while current mission-bound authorization drafts address authorization as a separate enforcement problem [13, 14].

6.1 Resolver sketch

The runtime separates semantic interpretation from infrastructure authority. A model or deterministic planner may propose the required capability; the runtime filters, ranks and binds providers deterministically.

```
required = mission.required_capabilities(stage)
working = []

for capability in required:
    declared = registry.find(capability, tenant=mission.tenant)
    semantic = semantic_match(declared, capability)

    admissible = [p for p in semantic if
        tenant_valid(p, mission) and
        authority_valid(p, mission) and
        policy_valid(p, mission) and
        declaration_signature_valid(p) and
        attestation_valid_if_required(p) and
        schema_version_valid(p) and
        transaction_compatible(p, mission, stage) and
        residency_valid(p, mission) and
        outcome_verifiable_if_required(p)]

    if not admissible:
        return BLOCKED(capability)

    selected = deterministic_rank(admissible,
        availability, SLO, latency, cost, audit_quality)

    binding = hash(mission_id, stage, capability,
        selected.provider_id, selected.version,
        selected.declaration_digest,
        approved_parameter_digest,
        current_policy_digest,
        authority_chain_digest)

    working.append(binding)

expose_to_model(stage_capabilities(working))   # MCWS only
execute_only_if(binding_revalidates_at_point_of_use)
observe_authoritative_state()
recompose_if_provider_policy_or_mission_state_changed()
```

This is deliberately conservative software architecture. The model is not allowed to choose a weaker-audit provider merely because that provider is technically admissible; ranking policy is part of the deterministic runtime and is itself auditable.

6.2 Outcome verification is a runtime pattern, not a new protocol

For consequential operations, R1.3 recommends carrying a local verification descriptor alongside the selected provider: an expected business-state delta, an authoritative readback capability, a correlation/idempotency key, a verification window, partial-success semantics and (where the business process supports it) a compensation capability. A successful tool receipt is therefore evidence of invocation, not automatically evidence that the

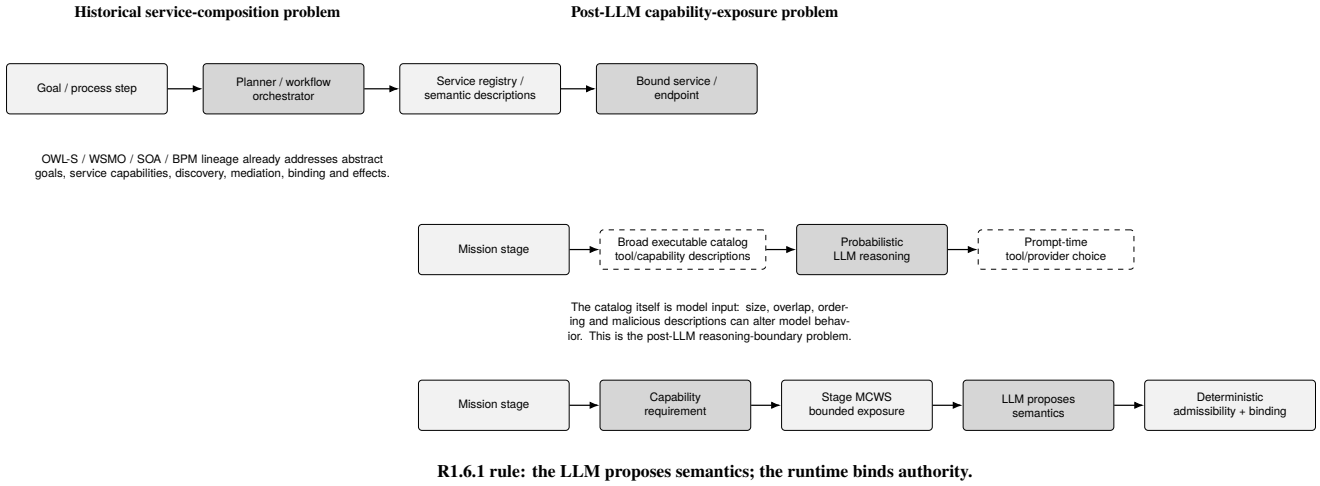


Figure 3. The historical and post-LLM problems are related but not identical. Classical service composition resolves abstract intent to services; an LLM-era runtime must additionally control the probabilistic reasoning surface created by executable capability descriptions. MCWS addresses this reasoning-boundary problem while deterministic runtime logic owns admissibility and binding.

business mission completed. This descriptor is a **deployment pattern**; it is not proposed as an MCP extension or a new standards contract, and the frozen experiment did not independently validate production readback semantics.

7 SAP S/4HANA worked example

SAP Integration Suite now supports MCP servers created from existing API artifacts, OpenAPI HTTP endpoints and RFC-based SAP S/4HANA/ECC backends. RFC-based servers can discover selected RFC functions and expose them as MCP tools; the resulting MCP artifact is versioned, deployable, policy-protected and observable [9, 17, 18].

Consider a procurement mission:

> **Mission:** diagnose why a purchase order is blocked; if a consequential release/change is authorized, perform the approved operation and verify the resulting S/4HANA business state.

The architectural value is not "LLM calls BAPI." SAP already supplies the bridging machinery. The value is that the mission runtime can keep `purchase_order.read` stable while choosing an RFC-based or API-based MCP provider according to compatibility, policy and health. SAP's documentation explicitly supports RFC-backed MCP for S/4HANA and ECC and documents purchase-order BAPIs including `BAPI_PO_GETDETAIL1`; the exact tool set exposed by an enterprise remains deployment-specific [9, 19].

7.1 A real full-stack proof trace already exercised

A separate frozen Capella/A4Layer proof demonstrates that the architecture is not limited to a paper diagram. The historical path reached:

Capella → RAP backend → SAP activation → service publication → OData V4 → Fiori Elements → BAS build → `reentranceTicket` SSO → SAP UI5 ABAP Repository deployment → Application Index → rendered real OData.

The frozen milestone is recorded as the first real Capella full-RAP end-to-end pass, with application `ZA4CLANG0010`. The execution boundary was explicit in the engineering evidence: Capella orchestrates, A4Layer executes, SAP is the system authority, and BAS is the frontend engineering runtime. Exact machine identifiers are retained in Appendix C. This matters because the whitepaper's runtime pattern did not emerge from an abstract "LLM calls tools" model; it emerged from trying to preserve authority and lifecycle state across a real enterprise development chain.

> **POC interpretation.** The ZA4CLANG0010 flow is implementation proof for the surrounding execution architecture, not an extra row in the Broad/Static/MCWS experiment. The scientific result and the engineering proof remain separate evidence streams.

8 Adoption roadmap: from Broad exposure to mission-aware execution

A practitioner should not begin by replacing the entire agent platform. A five-stage migration is safer.

9 Identity and policy: how much control is enough?

Not every tool call needs the same policy machinery. The following matrix is intentionally pragmatic.

AWS AgentCore Policy and Google Agent Gateway already demonstrate deterministic gateway enforcement as a real product pattern rather than a theoretical requirement [7, 6].

Table 4. Illustrative SAP mission flow. Tool names beyond the read example are deployment-specific; the flow is architecture, not an experiment reported by this paper.

Stage	Required capability	Possible SAP implementation	Runtime control	Outcome
1	Read purchase-order detail	Integration Suite RFC-based MCP exposing `BAPI_PO_GETDETAIL1` or an approved Purchase Order API	Read-only capability; actor/tenant scope; current provider health	PO detail, release state, blocking context
2	Resolve release/approval prerequisites	SAP API/RFC tools plus business policy source	MCWS adds only prerequisite capabilities; deterministic policy/approval decision remains outside model	Authorized/blocked next action
3	Execute approved change/release	Approved S/4HANA write API/RFC exposed behind MCP gateway	Consequential-write policy, identity, idempotency/commit semantics, optional human approval	Execution receipt / business document response
4	Verify business state	Readback via S/4HANA system-of-record capability	Correlation key + expected state delta + verification window	VALIDATED / MISMATCH / PARTIAL / BLOCKED
5	Recompose on provider failure	Alternate prequalified SAP API/RFC provider	Mission-relative substitutability and source/version checks	Mission continues without rewriting business intent

Table 5. Incremental adoption roadmap.

Stage	Operating model	Engineering move	Exit evidence
0	Broad exposure	Inventory agents, MCP servers, APIs and tool calls. Establish baseline task success, wrong-tool calls, tokens and latency.	Measured baseline; no hidden tool estate.
1	Governed catalog	Put MCP/tools behind registry/gateway; normalize ownership, version, health and identity.	Discoverability + policy/audit coverage.
2	Capability normalization	Map tool endpoints to provider-neutral business capability IDs; attach native annotations and existing declaration metadata rather than inventing new schemas.	Capability-to-provider map; incompatible duplicates identified.
3	Stage MCWS	Resolve only the capabilities needed for the current stage; retain deterministic state outside the model.	Reduced active tool surface; no loss of required capability coverage.
4	Runtime recomposition	Prequalify directional/degraded provider substitutability; react to provider health and mission-state changes.	Recovery/recomposition tests; explicit BLOCKED/DEGRADED state when no admissible substitute exists. Composite substitution remains unresolved architecture (Sec. 19).
5	Consequential outcomes	Add delegated authority, deterministic write policy, event/task handling, readback/compensation semantics.	Audit trail from mission intent to authoritative outcome.

10 Engineering failure modes observed in practice

Enterprise agent execution fails at boundaries that do not exist in toy tool-calling demos. Table 7 records failure classes preserved from engineering runs. These observations are not frequency estimates; they are design evidence for why mission state must include provider, lifecycle and system-of-record facts.

Why this matters. In enterprise systems, "tool selected" and "HTTP returned" are weak milestones. Real mission progress depends on provider health, platform semantics, object identity, publication/activation state, authentication context and authoritative readback.

11 Future-proofing MCWS as context windows grow

The strongest challenge to an MCWS-style argument is obvious: if frontier models can accept hundreds of thousands or millions of tokens, why minimize tool exposure at all? GPT-5, for example, publicly exposes a 400k-token context window [21]. The answer is that **context capacity and capability control are different problems**.

2026 tool-use research still reports wrong-tool calls, premature actions, semantic collision and selection degradation as tool catalogs scale. CMTF explicitly finds reliability and efficiency costs from larger menus; ToolScope reports sizable tool-selection gains from context-aware filtering; BiasBusters shows provider/ordering bias among functionally equivalent tools; Tool Forge argues that larger context does not remove token traffic, latency and cognitive noise [22, 23, 24, 11].

R1.6.1 separates what the frozen benchmark *measured* from what bounded exposure is intended to support. **Measured:** primary mission success/selection behavior under Broad, Static and MCWS, plus stage-visible exposure reduction as an implementation-conformance property. System-level token/time differences remain descriptive because the intervention changed more than one mechanism. **Architectural rationale, not independently measured:** least-capability exposure, attack-surface reduction, policy clarity, reasoning-boundary auditability and security against malicious metadata or prompt injection. These remain design objectives, not empirical outcomes of the frozen experiment.

Provider independence and runtime recomposition remain properties of the Mission–Capability–Provider indirection and binding layer, not of MCWS specifically. Thus, if context windows become effectively free and very large, the token-economy rationale weakens; the stage-scoping question remains relevant only to the extent that selection, governance or security benefits are separately demonstrated.

Table 6. Policy/identity control by operation class. Emerging specifications are shown as options, not mandates.

Operation class	Minimum control	Additional enterprise control	Why
Low-risk read	Authenticated MCP endpoint + allowlist	Basic role/scope filtering; rate limits	Prevent accidental estate-wide visibility.
Sensitive / tenant-scoped read	Workload/user identity + ABAC-style scope	Gateway policy, tenant context, audit, data classification	Identity and data boundary matter even without writes.
Consequential write	Strong caller identity + deterministic parameter policy	Human approval where required, idempotency, source/version binding, authoritative outcome verification	A valid tool call is not sufficient evidence of permitted or correct business change.
Delegated multi-agent action	Verifiable delegation/token exchange	Attenuated authority and actor chain; emerging AIP/mission-bound patterns may be relevant [20, 14]	Prevent authority expansion across agent hops.
Long-running/event mission	Stable workload identity + task correlation	Mission lifecycle, event subscription, temporal policy and revocation	Authority/state may change while work is in progress.

Table 7. Observed enterprise failure modes and the architecture lesson extracted from each.

Failure class	Observed engineering evidence	Runtime implication
Provider unavailable / unstable	An early multi-landscape pilot was confounded because the ADT MCP endpoint was not listening. A controlled replication added an ADT stability gate and completed with zero ADT-down samples and zero SAP writes.	Provider health is mission state. Historical capability proof cannot override current availability; fail closed or re-resolve.
SAP semantic validation	SAP rejected a transactional projection with "Transactional Projection View must be part of a business object." The issue was semantic RAP structure, not HTTP transport.	API/schema success is not system validity. Enterprise capability resolution must preserve platform lifecycle and semantic constraints.
Generated identity mismatch	A later compiler path initially assumed `ZPRODUCT3APP_O4`; the generator's actual object identity was `ZZUI_PRODUCT3APP_O4`.	Never guess enterprise object identity. Canonical manifests should be the naming authority; adapter-specific identities stay below the canonical model.
Successful deploy + tooling warning	BAS/SAP emitted literal "Deployment Successful." for `ZA4CLANG0010`; a later `ProxyType` exception occurred after deployment and was retained as a post-deploy tooling warning.	Lifecycle evidence needs precedence. A downstream warning must not overwrite an authoritative earlier success state.
Flavor-specific readiness	Managed RAP and read-only CDS paths did not expose identical readiness fields; treating a flavor-specific `backendContract` as universal caused a false UX blocker.	Readiness must derive from canonical compiler/execution state, not one UI-specific field.

12 What the experiment actually supports

The empirical study remains unchanged from the frozen technical report. After correcting the M003 Static grading asymmetry against the already-frozen post-change target, primary mission success was:

Table 8. Corrected primary mission success.

Architecture	Success	Rate
Broad	5/48	10.4%
Static	17/48	35.4%
MCWS	30/48	62.5%

At the 16-case cluster level, Broad–MCWS had a +52.08 percentage-point mean difference (Holm-adjusted $p = 0.00293$), Static–MCWS +27.08 points (Holm-adjusted $p = 0.03125$), while Broad–Static remained directional only (Holm-adjusted $p = 0.2266$). Held-out results were Broad 0/12, Static 2/12, MCWS 7/12 ($n = 4$ **independent held-out cases; descriptive/directional only**).

Do not overread the experiment. M002/M003 deliberately stress provider substitution and requirement recomposition. The stage-exposure mechanism was implemented and executed as designed, confirming implementation conformance rather than independently proving downstream security, collision, latency, cost or auditability benefits. H4 provider-substitution telemetry is illustrative mechanism validation (4 of 5 prequalified cases). The study does not establish universal MCWS superiority or production cost savings.

13 When to choose Static versus a mission-capability runtime

The benchmark supports the right-hand column under its designed dynamic stressors. It does not prove that every enterprise workload belongs there.

14 2026 platform map: buy the plumbing, build the mission semantics

The market is rapidly commoditizing registry, gateway, policy and observability. Enterprises should exploit that rather than rebuild it.

The practical design principle is simple: **buy or reuse the control-plane plumbing; build only the mission semantics that are specific to your business.**

Table 9. Architecture decision framework.

Condition	Prefer Static / compiled path	Prefer mission-capability runtime
Provider landscape	One stable provider; low churn	Multiple providers; failover/substitution required
Mission requirements	Stable, repetitive and known	Requirements become conditional or change mid-mission
Risk	Read-only / low consequence	Consequential writes, approval, compensation or readback
Tool estate	Small, distinct catalog	Large/redundant catalog or cross-system capability overlap
Execution duration	Short request-response	Long-running tasks/events where state or authority can change
Operations	Simplicity and deterministic throughput dominate	Resilience, provider neutrality and dynamic capability eligibility dominate

Table 10. Representative platform roles in a mission-capability architecture. This is a capability map, not a vendor ranking.

Platform	Useful existing role	What remains a mission-runtime concern
SAP Integration Suite	Convert API/OpenAPI/RFC-based SAP backends into versioned, policy-protected MCP servers; publish/catalog/monitor them [25, 18]	Provider-neutral capability identity, stage resolution, cross-provider substitutability, mission-state recomposition and outcome semantics.
MuleSoft Agent Fabric	Cross-vendor registry, discovery, governance, orchestration and observability; graph-based guided determinism [5, 26]	Mission-specific capability model and evidence-backed runtime policy for the enterprise's own business semantics.
Google Agent Registry/Gateway	Register MCP servers/tools/skills/agents; resolve endpoints; enforce IAM/policy/observability at gateway [6, 27]	Mission decomposition, MCWS and application-specific provider substitutability/outcome checks.
AWS AgentCore Gateway/Policy	MCP gateway plus real-time Cedar authorization over tool calls and parameters [28, 7]	Mission semantics and dynamic capability resolution above gateway policy.
Microsoft Agent 365	Central agent registry with synchronization from AWS, Google, Salesforce and Databricks [8]	Tool/capability resolution and enterprise workflow semantics.
IBM watsonx Orchestrate	Enterprise agent control plane for catalog, governance, operations and scale [12]	Provider-neutral mission model and capability-stage projection.

15 Build, buy and compose

The engineering POCs reinforce a pragmatic deployment boundary. Enterprises should not build a new agent platform merely to adopt the mission-capability pattern.

Practical rule. Buy the plumbing. Compose the providers. Build only the mission-semantic layer your enterprise uniquely needs.

16 Historical architecture ancestry: what is not new

The long-horizon thesis is intentionally constrained by two decades of prior systems work. OWL-S already modeled services with preconditions, outputs and effects and targeted automated discovery, invocation, composition and monitoring [29]. WSMO went closer still: it separated user *Goals* from Web-service *Capabilities*, described preconditions/postconditions/assumptions/effects, included non-functional properties such as financial, performance, reliability, security and trust, and used mediators to connect goals and services [30, 31]. Therefore **Mission → Capability → Provider** and semantic provider discovery are not claimed as new.

Durable execution and compensation are also established. Temporal persists workflow state and resumes after process/network failure; current AI-agent guidance explicitly separates deterministic durable workflows from non-deterministic LLM/tool activities [32, 33]. Saga patterns coordinate long-lived distributed transactions using continuation and compensating actions [34]. R1.6 therefore does not claim durable mission state or compensation as inventions.

Residual long-horizon question. What changes in the MCP/LLM era is not the existence of goals, workflows or service composition. The research question is whether a durable mission can continuously project a *stage-scoped capability requirement*, bind it to currently admissible providers under authority/trust/state constraints, expose only the active capability working set to probabilistic reasoning, observe authoritative outcome evidence, and then recompose when that evidence changes mission state.

The closest defensible architecture combination is consequently:

Durable Mission State → Capability Requirement → Stage MCWS → Admissible Provider Binding → Execution → Outcome Evidence → Recomposition

The frozen experiment evaluates only a subset of that loop: capability exposure/resolution and recomposition behavior under the authored provider landscapes. It does not validate the full long-horizon stack.

17 Closest prior art and residual contribution

R1.6.1 treats both historical and contemporary systems as close prior art rather than distant citations. Dynamic/minimal tool retrieval is **not** claimed as novel: ToolLLM paired tool-use training with API retrieval over a large API inventory; Gorilla coupled API generation with retrieval over changing API documentation; AnyTool used hierarchical retrieval to select candidates from a 16k+ API space; and ToolRet formalized large-scale tool retrieval as a critical first step in tool-using LLM pipelines [1, 2, 3, 4]. The residual question here is narrower: mission-stage-conditioned exposure inside an enterprise runtime that separates probabilistic

Table 11. Build/buy/compose boundary suggested by the engineering proofs.

Decision	Typical responsibility	R1.3 position
Buy / reuse	MCP gateway, registry, OAuth/workload identity, API lifecycle, policy enforcement, logs/metrics, SAP Integration Suite, cloud agent gateways	Commodity/control-plane capabilities should remain vendor or platform responsibilities where possible.
Build	Mission semantics, capability normalization, stage MCWS, provider substitutability policy, recomposition state, business outcome interpretation	These are enterprise-specific semantics and are the narrow layer this whitepaper argues teams may need to own.
Compose	ADT MCP, SAP APIs/RFC MCP, Fiori/Open UX tooling, UI5 validation, BAS, enterprise APIs/events and systems of record	The runtime should compose qualified providers; it should not absorb every provider's implementation knowledge into the LLM prompt.

semantic proposal from deterministic admissibility/binding and authoritative outcome observation.

Status of contemporaneous references. ACC and the Mission-Bound Authorization/Capability Binding materials are emerging proposals cited as representative of the design space, not ratified standards or stable runtime dependencies. Queen-Bee and Tool Forge are contemporaneous 2026 preprints used to bound novelty; they are not treated as established consensus. The architecture is intentionally written so that these artifacts can evolve or disappear without becoming load-bearing protocol dependencies.

After these boundaries, the defensible contribution is not generic goal-to-service composition. It is the agentic-runtime combination of: (i) durable mission state that is independent of model/provider sessions; (ii) a stage-scoped Mission Capability Requirement and MCWS reasoning boundary; (iii) an admissibility-aware Runtime Binding Record rather than prompt-time tool choice alone; (iv) outcome evidence that can trigger mission-state recomposition; and (v) a controlled, transparently corrected Broad/Static/MCWS comparison across heterogeneous provider landscapes. Each architectural component has ancestry; the contribution is the modern combination, evidence discipline, and practitioner blueprint.

18 Executive migration checklist

A CTO/enterprise architect can use the following as a pre-production gate:

1. Can every agent-visible endpoint be traced to an owned business capability and version?
2. Are read/write/destructive/idempotent semantics declared using native or established metadata rather than prompt prose alone?
3. Does business authorization remain in the system/gateway boundary rather than in model reasoning?
4. Can the runtime explain why a provider/tool was eligible for the active mission stage?
5. Is mission-relative provider substitutability prequalified before failover is needed, including degraded and composite cases?
6. Can an event/task resume with the correct identity, mission state and policy version?
7. Can a consequential action be correlated to authoritative readback and, where relevant, compensation?
8. Are Broad/Static/dynamic architectures evaluated on the enterprise's own workload distribution rather than assumed from this paper?

19 Long-horizon thesis: from tool access to portable missions and dynamic capability networks

R1.6 deliberately avoids predicting that today's protocol boundaries survive unchanged. MCP is the principal 2026 capability-access protocol used to ground the architecture, but it is treated as **replaceable protocol infrastructure**, not a permanent architectural center. The MCP roadmap already prioritizes agentic messaging, identity and enterprise-ready security [35, 36]; A2A already exposes Agent Cards, stateful tasks, streaming and push notifications [37, 38]. These demonstrate active protocol convergence, not a guarantee that "MCP = vertical" and "A2A = horizontal" will remain fixed for twenty years.

R1.6.1 hardened thesis. The mission—not the model, agent framework, or protocol session—should become the durable unit of autonomous enterprise computation. The runtime continuously converts mission state into a stage-scoped capability requirement, binds that requirement to providers that are *currently admissible* under semantics, authority, policy, trust, transactional behavior and outcome constraints, and updates/recomposes the mission from authoritative observed evidence.

The term *Autonomous Capability Network* remains a long-horizon scenario label, not a standards or novelty claim. Its usefulness depends on prerequisites such as portable semantics, delegated authority, provider attestation, durable mission state, liability/trust federation and deterministic safety boundaries.

19.1 Maturity horizons conditioned on prerequisites

Fixed calendar dates invite false precision. R1.6 therefore replaces the earlier 2030/2035/2045 schedule with maturity horizons whose transition depends on precursor capabilities.

19.2 Operational schema: four runtime records

Figure 4 incorporates the senior-review boundary. The architecture no longer treats a generic "capability contract" as one object. It separates demand, supply, binding and evidence:

1. **Mission Capability Requirement**: what the current mission stage requires, independent of provider identity.
2. **Provider Capability Declaration**: what a provider claims to supply, sourced from MCP/tool metadata, ACC-like declarations, enterprise catalogs, APIs or other protocols.

Table 12. Closest 2026 systems/specification neighbors.

Work	What it already provides	Residual distinction of this paper
Queen-Bee Agents [10]	Capability retrieval, BeeSpec intermediate representation, tenant-scoped MCP connectors, governed execution; retrieval beats static on its task set	Overlapping scoped-execution design space. This paper differs in mission-stage framing, provider-landscape evaluation scope and the corrected Broad/Static/MCWS benchmark; it does not claim architectural priority.
Tool Forge [11]	Validation-carrying capability contracts/tool capsules; governed catalog; intent-scoped MCP router; 99%+ context reduction estimate	Overlapping governed catalog/routing space. This paper assumes capabilities exist and studies mission-stage resolution and evidence boundaries; it does not claim architectural priority.
Agent Capability Contract [13]	Portable business-capability declaration metadata for reach, risk, approval, subject and audit	R1.2 consumes such declarations; it does not claim a new contract standard.
Mission-Bound Authorization / Capability Binding [14, 15]	Mission as approved authorization object; point-of-use enforcement; capability-source digest binding	Complementary authorization layer. This paper's mission contract represents orchestration requirements and runtime capability need, not authorization issuance.
MuleSoft Agent Fabric [5]	Productized discovery, governance, orchestration and observability across agents/tools/MCP servers	Market proof that generic "fabric" is infrastructure vocabulary, not this paper's novelty.

Table 13. Prerequisite-conditioned evolution thesis rather than a calendar forecast.

Horizon	Architectural transition	Required precursor	Classification
H0	Remote tool/capability connectivity	MCP/API adapters, stable schemas, basic discovery	**CURRENT**
H1	Governed enterprise capability access	Registry/gateway, identity, policy, ownership/versioning	**CURRENT / EMERGING**
H2	Mission-relative provider substitution	Portable capability semantics, provider declarations, transaction/outcome compatibility metadata	**ARCHITECTURE HYPOTHESIS**
H3	Portable missions across runtime/model boundaries	Durable state, authority/delegation portability, idempotency and pending-task semantics	**ARCHITECTURE HYPOTHESIS**
H4	Federated cross-enterprise capability networks	Trust federation, liability/commercial frameworks, interoperable credentials and evidence	**LONG-HORIZON**
H5	Cyber-physical missions	Digital-twin/simulation layers, attestation and certified deterministic safety envelopes	**LONG-HORIZON**

3. **Runtime Binding Record**: which concrete provider/version was selected, why it was admissible now, and which constraints/idempotency semantics apply.
4. **Outcome Evidence Receipt**: what authoritative enterprise/physical state was subsequently observed; this is distinct from a declared effect or transport success code.

19.3 Mission portability is more than durable execution

ARCHITECTURE HYPOTHESIS. Temporal and similar durable-workflow systems already preserve workflow state and resume after failure [32, 33]. R1.6 therefore uses *mission portability* more narrowly: state remains meaningful when cognition, runtime, provider or protocol changes. A portable mission needs more than a checkpoint. It must carry the mission contract, current stage, satisfied/pending capability obligations, evidence, delegated-authority context, policy snapshot, binding history, execution/idempotency state, asynchronous tasks, budget/deadline state, outcome evidence and outstanding compensation obligations.

A portable policy snapshot is historical evidence, not portable executable authority. When a mission resumes under another runtime, policy version or provider binding, pending consequential work must be re-authorized against the *current* policy and authority state. In compact form:

$$\text{HistoricalPolicySnapshot} \neq \text{CurrentExecutionAuthority}.$$

This ties portability to the same point-of-use revalidation rule used for TOCTOU and provider rebinding.

A consequence is that **models become cognitive capability providers rather than owners of mission state**. The enterprise may change a reasoning model without rewriting the business mission, provided the new provider satisfies the required cognitive capability and policy constraints. This is an architecture hypothesis, not a result of the 2026 study.

19.4 Mission-relative substitutability, including degraded and composite bindings

Two providers exposing the same verb are not necessarily interchangeable. R1.6 replaces symmetric "provider equivalence" with a directional relation:

$$P_j \preceq_{M,s} P_i$$

meaning that provider P_j is an admissible substitute for P_i for mission M at stage s. Substitution is allowed only when P_j satisfies the stage's required semantic effect, preconditions, authorization, transaction/settlement semantics, idempotency, compensation requirements, schema/version constraints, data residency/jurisdiction, trust/attestation threshold, SLO and outcome-verification requirement.

ARCHITECTURE HYPOTHESIS. The relation is intentionally directional and stage-relative. A read-only provider may be a degraded substitute for a richer provider during a diagnostic stage while remaining inadmissible for a write stage. The runtime may therefore enter an explicit `DEGRADED_CAPABILITY` state rather than pretending full equivalence. Entering that state does **not** inherit prior execution authority: the degraded binding and any changed effect/transaction semantics must be re-evaluated and re-authorized under current tenant, policy and delegated-authority state before execution.

A second open problem is **composite substitution**. One provider may satisfy a capability atomically while another requires an ordered set of providers or calls. A composite binding

$$B_{M,s} = \{P_1, \ldots, P_k\}$$

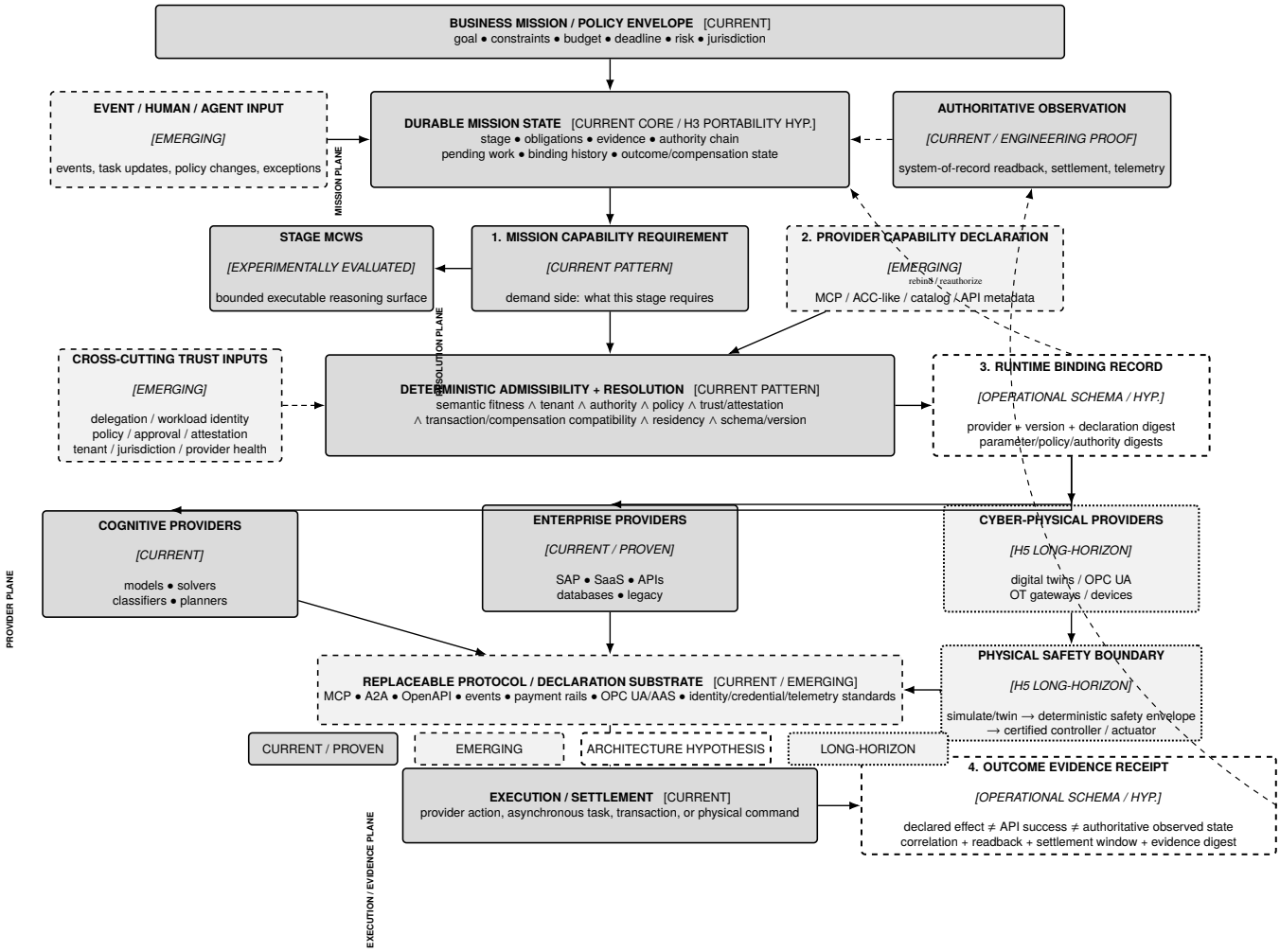


Figure 4. R1.6.1 maturity-coded dynamic architecture. Solid boxes are current/proven, dashed boxes are emerging or architecture-hypothesis elements, and dotted boxes are long-horizon. Mission state produces demand; provider declarations supply candidates; deterministic admissibility creates a binding; observation feeds recomposition. Cyber-physical providers remain behind an explicit deterministic safety boundary.

requires explicit ordering, intermediate-state semantics, atomicity/failure-window assumptions, idempotency, compensation and settlement criteria. A multi-step Saga-like path is not automatically equivalent to a single transactional call even if the nominal business verb is the same. **Composite substitution remains unresolved architecture: Saga-style compensation is not rollback.** Externally visible or non-compensable effects may require policy-driven forward recovery, operator intervention, blocking or an explicit degraded state. Current adoption guidance therefore applies first to qualified single-provider directional substitution; consequential composite bindings require independent qualification.

Admissibility is temporal. A provider that was valid at t_0 can become invalid before point of use because policy, credentials, health or jurisdictional constraints changed. Binding revalidation at execution is therefore part of the substitution mechanism, not a separate afterthought.

19.5 Threat model: discovery must not imply trust

A dynamic capability network increases attack surface. R1.6.1 therefore treats provider qualification as a security decision and adds a minimum threat model. **Evidence boundary:** the controls below are architecture requirements/reference design; they were not adversarially exercised by the frozen MCWS experiment and are not security-certification results.

The corresponding admissibility rule is intentionally stricter than semantic similarity:

$$\begin{aligned} Eligible(P,M,s) = {} & Sem(P,M,s) \wedge Tenant(P,M) \\ & \wedge Auth(P,M,s) \wedge Policy(P,M,s) \\ & \wedge SignedDecl(P) \wedge Trust(P,M,s) \\ & \wedge Tx(P,M,s) \wedge Data(P,M,s) \\ & \wedge Version(P,M,s). \end{aligned}$$

Only providers inside this set are ranked. Ranking is deterministic and policy-controlled; it is not delegated to the model.

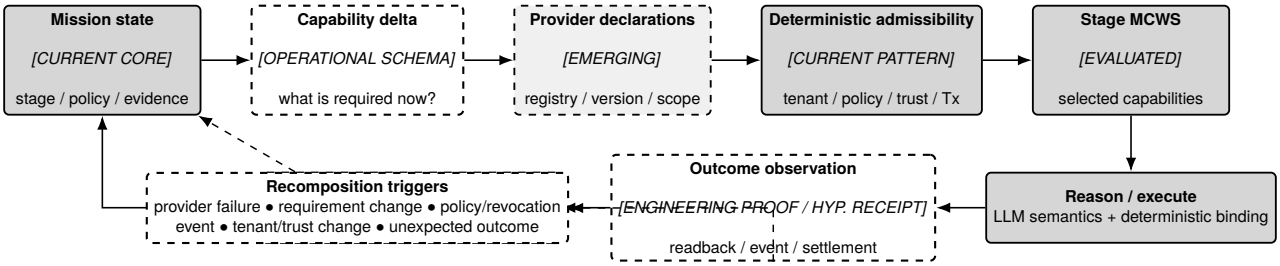


Figure 5. Maturity-coded dynamic recomposition loop. Current mission/MCWS mechanisms are separated from the operational-schema/hypothesis records used for portability and outcome evidence. Provider failure, policy/authority/tenant change, events or unexpected outcomes trigger deterministic re-resolution rather than prompt-level improvisation.

Table 14. Minimum R1.6 threat model and architectural response.

Threat	Failure shape	Required control
Registry / declaration poisoning	Provider advertises benign semantics while hiding effects or substituting metadata	Signed/versioned declarations, provider trust/allowlists, independent validation where risk warrants; metadata is never authority.
Provider substitution / confused deputy	Approved mission is rebound to a different source or broader authority	Cryptographically bind provider/source digest, mission stage and authority at point of use; attenuated delegation; re-authorize on rebind.
Parameter / binding mutation (TOCTOU)	Inputs, provider identity or policy state changes after approval	Bind parameter digest + provider/version + declaration digest + policy snapshot into the Runtime Binding Record; revalidate at execution.
Cross-tenant capability leakage	Capability metadata, cache entries or credentials from tenant A become visible in tenant B's MCWS	Tenant-scoped discovery/MCWS, tenant-bound credentials and binding records, tenant-aware cache keys; no cross-tenant capability cache without explicit federation.
Adversarial selection inside admissible set	Compromised model chooses the weakest-audit or highest-risk provider even though several providers pass eligibility	Model may propose capability intent; deterministic runtime owns provider ranking under signed/auditable ranking policy.
Result / readback falsification	Tool reports success or observer lies about final state	Separate invocation receipt from authoritative observation; use an independent trust domain/readback path for high-consequence operations.
Credential replay / authority amplification	Downstream actor receives broader authority than delegated or reuses stale authority	Short-lived scoped credentials, delegation chain, revocation and least-authority enforcement; current policy rechecked on resume/rebind.
Compromised runtime / model	Planning, binding or execution logic is malicious or corrupted	Deterministic policy boundary, auditable binding/evidence records, isolation/attestation for high-consequence workloads; safety-critical control stays outside the LLM.
Economic/reputation manipulation	Provider games price/reputation to win routing	Out of current proof scope; bounded catalogs/procurement policy now, stronger commercial trust mechanisms required before H4 economic routing.

19.6 MCWS after million-token context windows

Large context weakens any thesis that MCWS exists primarily to save tokens. GPT-5-class models already expose 400k-token context windows and Gemini documentation describes 1M-token or larger contexts [21, 39]. R1.6 therefore treats token efficiency as secondary. The long-lived MCWS rationale is: least-capability exposure, tool-selection precision, attack-surface reduction, authority/policy clarity, a bounded reasoning surface and auditability. Provider independence and deterministic recomposition are properties of the surrounding binding/runtime layer. If future models can reason perfectly over unlimited tool catalogs without security or selection cost, the exposure-reduction component should shrink in importance; the mission/binding/evidence architecture can still remain useful.

19.7 Delegated identity, verifiable credentials and attestation

CURRENT / EMERGING. SPIFFE supplies workload identity across dynamic heterogeneous systems and organizational boundaries [40]; W3C Verifiable Credentials 2.0 provides cryptographically secure, machine-verifiable credential data models [41]; confidential-computing work is exploring agentic workloads and remote attestation [42, 43]. These are building blocks, not proof of a unified future agent-identity stack.

LONG-HORIZON. A high-consequence capability request may ultimately require three independent proofs: *what workload is executing?*, *what qualifications/attributes does it hold?*, and *what authority has the principal delegated for this mission?*. Recomposition must not amplify that authority.

19.8 Reactive missions and long-running work

CURRENT / EMERGING. CloudEvents standardizes event envelopes [44]; A2A supports stateful tasks, streaming and push notification for long-running operations [37, 38]; Temporal demonstrates durable long-running agent/workflow state [32, 33]. A future runtime should therefore correlate events to durable mission state rather than require an open chat session:

$$\text{Enterprise Event} \rightarrow \text{Mission Create/Resume} \rightarrow \text{Capability Requirement} \rightarrow \text{MCWS} \rightarrow \text{Binding} \rightarrow \text{Execution/Observation}$$

19.9 Outcome evidence versus declared effects and transaction patterns

OWL-S/WSMO already modeled expected postconditions/effects [29, 30]; Saga architectures already model progress and compensation [34]. The useful agentic distinction is therefore not "effects are new." R1.6 separates four states:

Declared Effect ≠ API/Protocol Success ≠ Execution Receipt ≠ Authoritative Observed Business State

An *Outcome Evidence Receipt* is useful only if it records the correlation between mission/binding, expected state delta, observation source, observed state, time/settlement window and evidence digest. OpenTelemetry can contribute telemetry vocabulary [45], but telemetry alone is not proof of business settlement. The frozen experiment does not validate this production mechanism.

19.10 Economic routing is an emerging scenario, not a forecast

EMERGING. AP2 already defines an agent-payment protocol that can extend A2A and MCP [46], making budget and billing technically relevant to capability selection. **LONG-HORIZON**. Open cross-enterprise capability marketplaces, reputation-based routing and autonomous commercial negotiation remain speculative because liability, fraud, tax, procurement, vendor risk, contract law and dispute resolution are not solved by a protocol. R1.6 therefore classifies economic capability routing as **plausible long-horizon**, not expected evolution.

19.11 Cyber-physical providers require a deterministic safety boundary

EMERGING. OPC Foundation work is making hundreds of OPC UA Companion Specifications AI/MCP-ready [47], and Asset Administration Shell specifications provide standardized digital-twin models/APIs/security [48]. This supports the plausibility of semantic access to industrial assets.

R1.6 explicitly rejects a direct "LLM → MCP → robot" control path for consequential actuation. The safe reference boundary is:

Mission reasoning → Digital Twin / Simulation → Deterministic Safety Envelope → Certified Controller / Actuator → Physical Observation

The AI runtime can propose, simulate and select capabilities; safety-certified deterministic control remains authoritative for physical actuation.

19.12 Protocol pluralism and replaceability

The architecture assumes that protocols evolve or converge. MCP, A2A, event transports, payment protocols, OpenAPI, industrial protocols, identity/credential standards and future protocols are all adapters below the mission/runtime semantics. If A2A grows tool access, MCP grows agent messaging, or a new protocol replaces both, the four runtime objects above should remain meaningful. Protocol independence is therefore a **design constraint to be tested**, not an empirically demonstrated property and not a prediction about protocol winners.

20 A buildable research program toward the long-horizon thesis

The long-horizon view is useful only if it can be decomposed into falsifiable engineering steps. R1.6 therefore proposes the following POC sequence. These are future work items, not completed proofs.

Research discipline. R1.6 does not use future POCs to strengthen the existing MCWS result. Each POC should receive its own prospective contract, authority boundary, freeze and evidence manifest.

21 Limitations and future work

This whitepaper does not turn the technical report into a production proof, and the long-horizon architecture thesis is explicitly non-empirical. Broad architectural novelty is deliberately modest: goal/capability mediation, dynamic binding, durable workflow state, compensation and semantic service routing are historical prior art, while LLM-era dynamic tool retrieval/filtering also predates this release. The paper's narrower contribution is the bounded MCWS benchmark, enterprise engineering proof layer and disciplined operational synthesis around deterministic binding and observed outcomes. The empirical study uses one local model configuration, four authored mission families, read-only enterprise-flavored conditions and deterministic custom providers. The common-target reanalysis was post-audit and made zero new inference calls: the affected persisted Static outputs were regraded against the same already-frozen post-change target used for Broad and MCWS. The frozen study also lacks a size-matched control that holds visible capability count/token volume approximately constant while varying selection logic; it therefore cannot isolate mission-stage relevance from the generic benefit of a smaller non-adversarial subset. The SAP scenario is a reference architecture example, not an evaluated end-to-end transaction. ACC, AIP and mission-bound authorization references are emerging specifications/research and may evolve. Vendor capabilities change quickly; the platform table is a dated architectural map, not procurement advice.

The next empirical work should test the mission-capability runtime on: (i) larger and hosted models; (ii) real enterprise integration catalogs; (iii) asynchronous tasks/events; (iv) policy-enforced consequential writes; and (v) outcome verification under eventual consistency. Crucially, future work should separately measure selection quality, policy correctness, recovery, latency and cost rather than using token reduction as a proxy for all value.

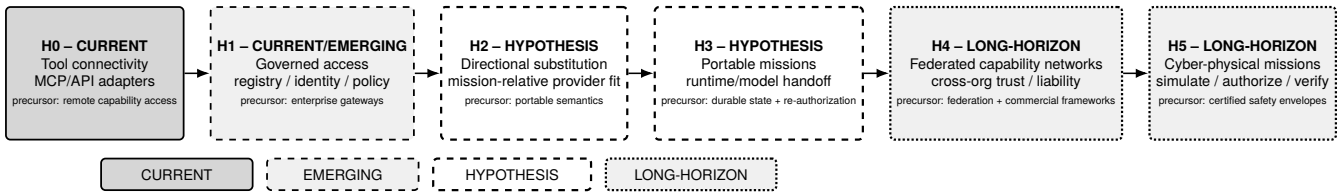


Figure 6. Prerequisite-conditioned horizons with explicit maturity encoding. CURRENT/EMERGING capabilities are separated visually from HYPOTHESIS and LONG-HORIZON scenarios; no calendar date is implied.

Table 15. Proposed next POC sequence. Each step can fail independently without invalidating the frozen 2026 experiment.

POC	Question	Minimal demonstrator	Evidence gate
R2	Capability routing	Two mission-admissible providers selected by health, SLO, latency, cost and policy	Same mission contract; deterministic provider choice; directional/degraded substitution rules; failover without prompt rewrite
R3	Portable mission state	Start under model/runtime A; resume under B	State hash continuity; no loss of satisfied/pending capabilities or authority context
R4	Delegated authority	Human → agent → sub-agent → SAP read/write boundary	Attenuation proven; no authority amplification; revocation observable
R5	Event-driven mission	SAP/Event Mesh/CloudEvents creates or resumes a mission	Event correlation; idempotent resume; authority/policy rechecked at resume
R6	Outcome receipt	Consequential action + authoritative SAP readback	Expected delta matched to system-of-record observation with immutable evidence receipt
R7	Economic routing	Provider choice constrained by budget/price/SLO	Cheapest qualified provider chosen without violating policy/quality threshold
R8	Digital-twin mission	SAP + AAS/OPC UA read-only simulation path	Model-independent mission state spans enterprise + operational digital twin
R9	Cyber-physical closed loop	Simulate → approve → execute → observe	Hard safety gate tied to applicable functional-safety practice (e.g., IEC 61508/61511 class where relevant), physical sandbox, compensation/recovery evidence

22 Conclusion

R1.6.1 is intentionally built on five layers rather than one inflated novelty claim. **First, credible lineage:** OWL-S, WSMO, SOA/BPM, Saga and durable-workflow systems already established goal/capability mediation, semantic service descriptions, dynamic binding, compensation and persistent execution state. This paper does not relabel those ideas as new. **Second, a measurable post-LLM problem:** executable capability descriptions can become part of a probabilistic model's active reasoning environment, so the exposed catalog itself can alter selection behavior, attack surface and auditability. The stage-scoped MCWS is the paper's evidence-backed mechanism for this reasoning-boundary problem.

Third, real experiment: the frozen corrected study remains the scientific anchor. MCWS achieved 30/48 primary mission successes (62.5%), Static 17/48 (35.4%) and Broad 5/48 (10.4%); Broad–MCWS and Static–MCWS remain significant after Holm adjustment while Broad–Static is directional only. These results apply to the evaluated benchmark and do not validate federation, economics, identity portability or cyber-physical safety.

Fourth, enterprise engineering proof: separate SAP/MCP/Capella/BAS POCs demonstrate that mission contracts, live MCP access, deterministic orchestration boundaries, SAP activation/readback and full-stack consumer paths are not merely diagrams. The failure register further shows why enterprise execution depends on provider health, lifecycle semantics, identity, activation/publication state and authoritative readback rather than on tool selection alone.

Fifth, disciplined architecture and roadmap: the Mission-Capability Runtime is presented as a practitioner pattern, not a new computational paradigm. Mission demand, provider declaration, runtime binding and outcome evidence are operational records; provider substitution is directional, stage-relative and potentially composite; model reasoning does not own final provider ranking or authority; tenant/policy/binding integrity is checked at point of use; and protocol independence remains a design constraint. Portable missions, trust federation, economic routing and cyber-physical execution remain prerequisite-conditioned hypotheses.

The resulting thesis is therefore narrower but stronger: **stage-scoped capability exposure addresses a post-LLM agent-tool-catalog problem already recognized in retrieval/dynamic-selection research; in this frozen benchmark MCWS achieved higher primary mission success than Broad and Static, but the study does not isolate mission-stage relevance from the generic benefit of a smaller visible subset. R1.6.1 pairs that bounded evidence with a deterministic practitioner runtime pattern for admissibility, binding and authoritative observation rather than claiming the full stack was empirically validated.** Whether the long-horizon roadmap becomes reality is a future research question; R1.6.1 makes that uncertainty explicit rather than borrowing confidence from the 2026 experiment.

References

[1] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun. "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs". In: *arXiv preprint arXiv:2307.16789* (2023). URL: https://arxiv.org/abs/2307.16789.

[2] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez. "Gorilla: Large Language Model Connected with Massive APIs". In: *arXiv preprint arXiv:2305.15334* (2023). URL: https://arxiv.org/abs/2305.15334.

[3] Y. Du, F. Wei, and H. Zhang. "AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Calls". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 11812–11829. URL: https://proceedings.mlr.press/v235/du24h.html.

[4] Z. Shi, Y. Wang, L. Yan, P. Ren, S. Wang, D. Yin, and Z. Ren. "Retrieval Models Aren't Tool-Savvy: Benchmarking Tool Retrieval for Large Language Models". In: *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 2025, pp. 24497–24524. DOI: 10.18653/v1/2025.findings-acl.1258. URL: https://aclanthology.org/2025.findings-acl.1258/.

[5] MuleSoft. *MuleSoft Agent Fabric Overview*. 2026. URL: https://docs.mulesoft.com/general/agent-fabric-overview (visited on 08/31/2026).

[6] Google Cloud. *Agent Registry Overview and Manage MCP Servers and Tools*. Aug. 26, 2026. URL: https://docs.cloud.google.com/agent-registry/overview (visited on 08/31/2026).

[7] Amazon Web Services. *Getting Started with Policy in AgentCore*. Amazon Bedrock AgentCore Documentation. 2026. URL: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy-getting-started.html (visited on 08/31/2026).

[8] Microsoft. *Registry Sync in the Microsoft 365 Agent Registry (Preview)*. May 21, 2026. URL: https://learn.microsoft.com/en-us/microsoft-agent-365/admin/agent-registry (visited on 08/31/2026).

[9] SAP SE. *Create an MCP Server by Referring to an RFC-Based Backend*. SAP Help Portal – Integration Suite. 2026. URL: https://help.sap.com/docs/integration-suite/isuite-integrations-and-apis/create-mcp-server-by-referring-to-rfc-based-backend (visited on 08/31/2026).

[10] D. Zhang and Liaotian. "Queen-Bee Agents: A BeeSpec-Centered Architecture for Governed Enterprise MCP Orchestration". In: *arXiv preprint arXiv:2606.06545* (2026). URL: https://arxiv.org/abs/2606.06545.

[11] S. Rao. "Tool Forge: A Validation-Carrying Toolchain for Governed Agentic Execution". In: *arXiv preprint arXiv:2605.28000* (2026). DOI: 10.48550/arXiv.2605.28000. URL: https://arxiv.org/abs/2605.28000.

[12] IBM. *Agentic Control Plane in IBM watsonx Orchestrate*. July 2, 2026. URL: https://www.ibm.com/new/announcements/introducing-the-agentic-control-plane (visited on 08/31/2026).

[13] Agent Capability Contract Project. *Agent Capability Contract v1 Specification*. Emerging open specification; early adoption. 2026. URL: https://agentcapability.org/docs/spec/ (visited on 08/31/2026).

[14] K. McGuinness. *Mission-Bound Runtime Enforcement*. Aug. 27, 2026. URL: https://mcguinness.github.io/mission-bound-authorization/draft-mcguinness-mission-runtime.html (visited on 08/31/2026).

[15] K. McGuinness. *Mission Capability Binding*. Aug. 27, 2026. URL: https://mcguinness.github.io/mission-bound-authorization/draft-mcguinness-mission-capability-binding.html (visited on 08/31/2026).

[16] Model Context Protocol Core Maintainers. *Tool Annotations as Risk Vocabulary: What Hints Can and Can't Do*. Mar. 16, 2026. URL: https://blog.modelcontextprotocol.io/posts/2026-03-16-tool-annotations/ (visited on 08/31/2026).

[17] SAP SE. *Configure an MCP Server Created from an RFC-Based Backend*. SAP Help Portal – Integration Suite. 2026. URL: https://help.sap.com/docs/integration-suite/isuite-integrations-and-apis/govern-and-manage-mcp-server-created-by-referring-to-rfc-based-backend (visited on 08/31/2026).

[18] SAP SE. *Developer Hub*. SAP Help Portal – Integration Suite. 2026. URL: https://help.sap.com/docs/integration-suite/sap-integration-suite/developer-hub (visited on 08/31/2026).

[19] SAP SE. *Create Purchase Order - Advanced / Purchase Order BAPIs*. SAP Help Portal. 2026. URL: https://help.sap.com/docs/PRODUCT_ID/af9ef57f504840d2b81be8667206d485/28f492016a52485784d85b7a31f7542d.html (visited on 08/31/2026).

[20] S. Prakash. "AIP: Agent Identity Protocol for Verifiable Delegation Across MCP and A2A". In: *arXiv preprint arXiv:2603.24775* (2026). URL: https://arxiv.org/abs/2603.24775.

[21] OpenAI. *GPT-5 Model*. 2026. URL: https://developers.openai.com/api/docs/models/gpt-5 (visited on 08/31/2026).

[22] R. S. Babu and L. G. Iyer. "ToolChoiceConfusion: Causal Minimal Tool Filtering for Reliable LLM Agents". In: *arXiv preprint arXiv:2606.06284* (2026). URL: https://arxiv.org/abs/2606.06284.

[23] M. M. Liu, D. Garcia, F. Parllaku, V. Upadhyay, S. F. A. Shah, and D. Roth. "ToolScope: Enhancing LLM Agent Tool Use through Tool Merging and Context-Aware Filtering". In: *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. 2026. DOI: 10.18653/v1/2026.acl-long.1573. URL: https://aclanthology.org/2026.acl-long.1573/.

[24] T. Blankenstein, J. Yu, Z. Li, V. Plachouras, S. Sengupta, P. H. S. Torr, Y. Gal, A. Paren, and A. Bibi. "BiasBusters: Uncovering and Mitigating Tool Selection Bias in Large Language Models". In: *International Conference on Learning Representations (ICLR)*. 2026. URL: https://www.microsoft.com/en-us/research/publication/biasbusters-uncovering-and-mitigating-tool-selection-bias-in-large-language-models/.

[25] SAP SE. *How Model Context Protocol (MCP) Server Enables AI Integration*. SAP Help Portal – Integration Suite. 2026. URL: https://help.sap.com/docs/integration-suite/isuite-integrations-and-apis/how-model-context-protocol-mcp-enables-ai-integration-with-apis (visited on 08/31/2026).

[26] MuleSoft. *Agent Fabric Release Notes*. 2026. URL: https://docs.mulesoft.com/general/agent-fabric-release-notes (visited on 08/31/2026).

[27] Google Cloud. *Resolve Endpoints and Build Orchestrators*. Aug. 19, 2026. URL: https://docs.cloud.google.com/agent-registry/resolve-endpoints-and-build-orchestrators (visited on 08/31/2026).

[28] Amazon Web Services. *Use an AgentCore Gateway*. Amazon Bedrock AgentCore Documentation. 2026. URL: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/gateway-using.html (visited on 08/31/2026).

[29] World Wide Web Consortium. *OWL-S: Semantic Markup for Web Services*. W3C Member Submission. Nov. 22, 2004. URL: https://www.w3.org/submissions/OWL-S/ (visited on 08/31/2026).

[30] World Wide Web Consortium. *Web Service Modeling Ontology (WSMO)*. W3C Member Submission. June 3, 2005. URL: https://www.w3.org/submissions/WSMO/ (visited on 08/31/2026).

[31] World Wide Web Consortium. *Web Service Modeling Ontology Primer*. W3C Member Submission. June 3, 2005. URL: https://www.w3.org/submissions/WSMO-primer/Overview.html (visited on 08/31/2026).

[32] Temporal Technologies. *Temporal Platform Documentation: Durable Execution*. 2026. URL: https://docs.temporal.io/ (visited on 08/31/2026).

[33] Temporal Technologies. *AI Agent Reference Architecture*. 2026. URL: https://go.temporal.io/platform-hub/ai-engineering/ai-reference-architecture (visited on 08/31/2026).

[34] Amazon Web Services. *Saga Patterns*. AWS Prescriptive Guidance. 2026. URL: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/saga-patterns.html (visited on 08/31/2026).

[35] D. Soria Parra and D. Delimarsky. *The New MCP Roadmap*. Model Context Protocol. Aug. 22, 2026. URL: https://blog.modelcontextprotocol.io/posts/mcp-roadmap/ (visited on 08/31/2026).

[36] D. Soria Parra. *The 2026 MCP Roadmap*. Model Context Protocol. Mar. 9, 2026. URL: https://blog.modelcontextprotocol.io/posts/2026-mcp-roadmap/ (visited on 08/31/2026).

[37] A2A Protocol Project. *A2A Protocol Core Concepts*. 2026. URL: https://a2a-protocol.org/latest/topics/key-concepts/ (visited on 08/31/2026).

[38] A2A Protocol Project. *Streaming and Asynchronous Operations*. 2026. URL: https://a2a-protocol.org/latest/topics/streaming-and-async/ (visited on 08/31/2026).

[39] Google. *Gemini API: Long Context*. June 22, 2026. URL: https://ai.google.dev/gemini-api/docs/long-context (visited on 08/31/2026).

[40] SPIFFE Project. *SPIFFE Overview*. 2026. URL: https://spiffe.io/docs/latest/spiffe-about/overview/ (visited on 08/31/2026).

[41] World Wide Web Consortium. *Verifiable Credentials 2.0 Family of Specifications Is Now a W3C Recommendation*. May 15, 2025. URL: https://www.w3.org/news/2025/the-verifiable-credentials-2-0-family-of-specifications-is-now-a-w3c-recommendation/ (visited on 08/31/2026).

[42] M. Bursell. *Protecting Agentic AI Workloads with Confidential Computing*. Confidential Computing Consortium. Jan. 20, 2026. URL: https://confidentialcomputing.io/2026/01/20/protecting-agentic-ai-workloads-with-confidential-computing/ (visited on 08/31/2026).

[43] Confidential Computing Consortium. *Confidential Computing Summit 2026: Six Signals From the Year the Foundation Got Real*. July 2, 2026. URL: https://confidentialcomputing.io/2026/07/02/confidential-computing-summit-2026-six-signals-from-the-year-the-foundation-got-real/ (visited on 08/31/2026).

[44] CloudEvents Project. *CloudEvents: A Specification for Describing Event Data in a Common Way*. 2026. URL: https://cloudevents.io/ (visited on 08/31/2026).

[45] OpenTelemetry Project. *OpenTelemetry Semantic Conventions 1.44.0*. 2026. URL: https://opentelemetry.io/docs/specs/semconv/ (visited on 08/31/2026).

[46] Google Cloud. *Powering AI Commerce with the Agent Payments Protocol (AP2)*. Sept. 16, 2025. URL: https://cloud.google.com/blog/products/ai-machine-learning/announcing-agents-to-payments-ap2-protocol (visited on 08/31/2026).

[47] OPC Foundation. *OPC Foundation Advances OPC UA for the AI Era with Companion Specifications Optimized for Agentic AI*. Apr. 20, 2026. URL: https://opcfoundation.org/news/press-releases/opc-foundation-advances-opc-ua-for-the-ai-era-with-companion-specifications-optimized-for-agentic-ai/ (visited on 08/31/2026).

[48] Industrial Digital Twin Association. *Asset Administration Shell Specifications*. 2026. URL: https://industrialdigitaltwin.org/en/content-hub/aasspecifications (visited on 08/31/2026).

A Scientific evidence capsule

Table 16. Frozen evidence used by the practitioner whitepaper.

Evidence item	Final interpretation
Broad vs Static	Static directional only; Holm-adjusted case-cluster $p = 0.2266$.
Broad vs MCWS	MCWS supported with benchmark boundary; Holm-adjusted $p = 0.00293$.
Static vs MCWS	MCWS higher in corrected primary case-cluster analysis; Holm-adjusted $p = 0.03125$.
Stage TER	Implementation-conformance evidence: stage exposure ran as designed; downstream security/collision/latency/auditability benefits were not independently measured.
Broad efficiency	Integrated architecture effect; Broad also changes model-planning mechanism and model-call count.
Provider substitution	Illustrative mechanism validation; independently gated mission success 4 of 5 prequalified cases.
Held-out split	Broad 0/12, Static 2/12, MCWS 7/12; $n = 4$ independent held-out cases; descriptive/directional only.
Raw-evidence status	Frozen; no R1.6 experiment reruns or raw-evidence mutations.

B Terminology boundary

- **Mission-Capability Runtime**: descriptive architecture term used by this whitepaper, not a proposed network protocol or standards claim.
- **MCP**: protocol substrate for capability discovery/invocation and related primitives.
- **MCWS**: stage-scoped set of capabilities active for the current mission stage in this study's architecture.
- **Capability declaration**: metadata about a business operation; may come from MCP annotations, ACC-like declarations, vendor catalogs or enterprise metadata.
- **Mission-bound authorization**: complementary authorization architecture; not synonymous with this paper's mission orchestration model.
- **Autonomous Capability Network**: scenario label only for a possible federated environment in which missions dynamically resolve governed cognitive, enterprise and physical capabilities. It is not a proposed standards name and not claimed as novel terminology.
- **Mission Capability Requirement**: demand-side statement of what the current mission stage needs; owned by the mission runtime.
- **Provider Capability Declaration**: supply-side provider metadata sourced from MCP/ACC/catalog/API/provider descriptions; a declaration is not authority.
- **Runtime Binding Record**: auditable record of the selected provider/version and the constraints under which the binding was admissible.
- **Outcome Evidence Receipt**: proposed evidence object linking an expected state change to authoritative observed state; not a standardized protocol object.
- **Mission portability**: architecture hypothesis that durable mission state can move across models/runtimes without losing authority, evidence or progress semantics.

C Engineering proof register

The following register makes the implementation claims auditable without pretending the POCs are one experiment. Exact identifiers are shown with breakable path formatting.

P1 **Provider-independent read-only mission.**
Frozen identifier: `SAP-READONLY-IMPLEMENTATION-PATH-001`. Target `TRL_EN`, client 100. Demonstrates that mission semantics and no-mutation constraints can be defined independently of provider identity. *Boundary: architecture/qualification mission; read-only.*

P2 **ADT-stable provider qualification.**
Frozen identifier: `EA4_R2_PRIMARY_REPLICATION_CLOSURE_R1`. Demonstrates a clean read-only 3x3 pilot under ADT MCP stability monitoring, with zero SAP writes. *Boundary: single-mission pilot; the later multi-mission program remains the primary science.*

P3 **Separated orchestration, execution and SAP authority.**
Frozen authority flags: `CAPELLA_ORCHESTRATES=true`, `A4LAYER_EXECUTES=true`, `SAP_IS_SYSTEM_AUTHORITY=true`. Demonstrates that orchestration, execution adapters and system-of-record authority can be separated in a working SAP engineering stack. *Boundary: engineering architecture proof, not a protocol standard.*

P4 **Full-stack SAP/Fiori consumer proof.**
Frozen milestone: `FIRST_REAL_CAPELLA_FULL_RAP_E2E=PASS`; application `ZA4CLANG0010`. Demonstrates RAP → activation → publication → OData V4 → Fiori → BAS → BSP deployment → rendered data. *Boundary: historical application is preserved and not reused.*

P5 **Bounded BAS executor.**
Frozen identifier: `CAPELLA_R6_1_RAP_TO_FIORI_UX_INTEGRATION_R3=PASS`. Demonstrates a four-operation command whitelist, zero automatic retry and explicit deployment-state classification. *Boundary: implementation proof; not arbitrary production-security certification.*

P6 **Scientific evidence authority.**
Frozen manuscript SHA-256: `E589393C6EEA0BA92E4EC521D1577A6F561F9ADFA38FF9EEE97A94695C666D90`. Contains the corrected Broad/Static/MCWS experiment and statistical interpretation. *Boundary: evidentiary authority for empirical claims.*

P7 **Observed enterprise failure semantics.**
Example run: `REQ-0618E1D1C75E466C`, plus preserved deployment and object-identity diagnostics. Demonstrates that SAP semantic validation, provider availability, generated identity and post-deploy state can fail independently. *Boundary: design evidence, not prevalence statistics.*

D Proof-state legend

- **PROVEN-LIVE**: exercised against a real enterprise/runtime target with captured evidence.
- **IMPLEMENTED**: implementation exists and was exercised, but the claim is not production-scale certification.
- **VALIDATED-READ-ONLY**: live/read-only evidence with mutation authority intentionally disabled.
- **EXPERIMENTALLY-EVALUATED**: evaluated within the frozen scientific program.
- **REFERENCE-DESIGN**: proposed architecture or deployment pattern not independently validated by this study.

E AI assistance and review provenance

GPT-based assistance, Claude Code and Gemini were used under author direction for adversarial review, source-level consistency checking, literature/market discovery, drafting and publication assembly. These reviews are internal AI-assisted red-team processes, not independent human peer review. All scientific judgments, freezes and final claims remain the author's responsibility.